Prisha Soam · Backend.

ARRIVO · INNOTECH'26 · CATEGORY 6 · YOUR STUDY GUIDE

Your part: the server. It is one Python file, `backend/server.py` (1,121 lines), and it does six jobs:

- stores every booking in a database
- refuses double bookings
- works out each bay's status
- pushes every change to every open screen live
- keeps people private without accounts
- serves the website itself

The rest of the team:

- **Vaibhav:** what you see in the browser
- **Kritika:** the map, lens and 3D
- **Krish:** the problem and impact

The good news. It's one file, with no libraries to install, and it reads top to bottom in 9 numbered sections. If you read it once slowly with this guide beside you, you will understand all of it.

Your slides:

- **11:** Tech stack. You present the **Backend** row after Vaibhav does the rest.
- **12:** Architecture: how a booking travels.
- **13:** Under the hood: four rules that keep it honest.
- **During the live demo:** when the second window can't book the same bay, you explain why.

How to use this guide. Read it twice. Then open `backend/server.py` and read it alongside, section by section. Run the server yourself (section 10) and try the commands.

1. Say this

> **Slide 11 · Backend row.** "The backend is one Python file using only Python's standard library: `http.server` for the web server, `sqlite3` for the database, `threading` so many people can use it at once, and Server-Sent Events for live updates. There's nothing to `pip install`. You run `python server.py` and it works."

Slide 12 · Architecture. "Here's one booking, start to end.

1. On your phone, you tap a free bay and book it. The browser sends the booking to our Python server.
2. The server checks for a clash and saves it in SQLite in one locked step. If someone else already has that time, it refuses with a 409 Conflict.
3. If it's saved, the server pushes the change to every open page over Server-Sent Events, so a friend's laptop shows that bay turn booked without refreshing."

Slide 13 · Under the hood. "Four rules keep it honest.

- **No double booking.** It's checked twice: inside a locked transaction in our code, and again by a trigger inside the database that refuses any two overlapping bookings on one bay.
- **Status from time.** A bay's status is worked out from the clock and the day's bookings, never typed in by hand, so it's always right.
- **Live without refresh.** Every page keeps one open connection; if it drops, it reconnects and catches up.
- **Private by default.** No sign-up. A random device ID marks your bookings, and we only store a hash of it. Other people's number plates are masked."

Live demo (double booking). "Two windows, two different people, same bay, same time. The first one booked it. The second can't. On screen the time is crossed out, but even if both pressed at the very same instant, the server takes the first and refuses the second with a 409. That happens inside a locked database step, and a database trigger would refuse it too."

2. The big picture

```
Phone / laptop browser                    backend/server.py (Python, port 8787)
SQLite file
--------------------                      ------------------------------------                ---
--------
GET  /api/lots/kiet-main/snapshot   ---->   read today's bookings ---------------------------->
bookings
POST /api/bookings                  ---->   check the rules, lock, check clash, save  -------->
bookings, events
                                    <----   201 Created + the booking   (or 409 Conflict)
GET  /api/lots/kiet-main/stream     ---->   stays open ... pushes "change" to every open page
GET  /  (and /assets/...)           ---->   sends the built website (frontend/dist)
```

One address does everything. `http://localhost:8787` gives you both the website and the API. The browser never talks to the database directly; only the server does.

What "standard library only" means

Python comes with a big set of built-in modules. We use only those. So:

- **No `pip install`.** Python 3.8 or newer, and it runs.
- **Nothing to break.** No outside library can change underneath us.
- **Easy to read.** Everything the server does is in one file you can read top to bottom.

MODULE	WHAT WE USE IT FOR
`http.server`	The web server: receives requests and sends replies (`ThreadingHTTPServer`, `BaseHTTPRequestHandler`).
`sqlite3`	The database.
`threading`	A thread per request, a lock, and the two background loops.
`queue`	One inbox per open page for live updates.
`json`	Reading and writing the data the browser sends and receives.
`secrets`	Secure random values: device IDs and booking codes.
`hashlib`	SHA-256, to store device IDs only as hashes.
`re`	Checking number plates and matching URLs.
`argparse`, `os`	Command-line options (`--port`, `--no-demo`) and settings.
`datetime`, `time`	Clock and dates.
`pathlib`, `urllib.parse`, `socket`, `errno`, `random`, `traceback`, `contextlib`	Files, URLs, network details, demo randomness, error printing.

3. The file, section by section

The file says this at the top, lines 15–24. Know the nine sections.

#	SECTION	LINES	WHAT IT DOES
1	Settings	48–67	Port 8787, database file, demo on/off, which websites may call the API.
2	The lot	70–112	Rows A–E, prices, bay types; builds the 96 bays. **Same rows as the frontend's `lot.jsx`: change one, change both.**
3	Database	115–313	Tables, the no-double-booking trigger, the lock, transactions, small query helpers.
4	Time and bay status	315–369	`slot_state()` (free / booked / ending / reserved / yours) and `mask_plate()`.
5	Seeding	372–459	Each day's "regular" background cars, the same for a given date every time.
6	Bookings	462–634	Make a booking, cancel one, check everything, and decide what each person may see.

#	SECTION	LINES	WHAT IT DOES
7	Live updates	637–660	Server-Sent Events: who's listening, and `publish()` to all of them.
8	Background loops	663–786	The 5-second clock (arrivals, departures), demo cars, reset.
9	The web server	788–1121	Every URL, the cookie, CORS, the live stream, serving the website, `main()`.

4. The database

SQLite is a whole database in **one file**: `backend/data/parkseat.db`. There's no separate database server to install. It's built into Python and used inside phones, browsers and planes. For one lot with a few hundred bookings a day, it's more than enough.

The tables (lines 121–206)

TABLE	ONE ROW IS…	IMPORTANT COLUMNS
`lots`	a parking lot	`id` = `kiet-main`, name, place
`rows`	a row in a lot	A–E, name, blurb, display order
`bays`	one bay	`A-14`, row, type, rate, size in metres
`bookings`	one booking	`bay_id`, `start_ms`, `end_ms`, `plate`, `status`, `code`, `owner`, `source`
`events`	one line of the live feed	time, bay, kind (`book`, `cancel`, `release`, `arrive`, `leave`)
`meta`	a setting	e.g. "this date has been seeded", the clock's last run

Booking columns worth knowing:

- **`start_ms`, `end_ms`:** times as **milliseconds since 1970**, exactly like JavaScript's `Date.now()`. The browser and server use the same numbers, so there's no time-format confusion. A booking holds its bay from `start` up to, **but not including**, `end`.
- **`status`:**
  - `confirmed`: holds the bay
  - `released`: left early; the end was moved earlier
  - `cancelled`: never used
- **`code`:** the gate code like `PS-7KQ2M`, which must be `UNIQUE`.
- **`owner`:** the **hash** of the device ID that made it. Empty for background and demo cars.
- **`source`:**
  - `api`: a real person booked it on the site
  - `seed`: a background "regular" car
  - `demo`: a demo car

Indexes (lines 165–167, 200) make lookups fast. A lookup like "bookings for bay A-14 around now" doesn't have to scan everything.

Settings when the database opens (`open_db`, lines 212–219):

- `journal_mode = WAL`: reading doesn't block writing, so the live map stays fast while bookings are saved.
- `foreign_keys = ON`: a booking can't point at a bay that doesn't exist.

The catalogue on every start. `sync_catalog()` (lines 261–276) writes the lot, rows and bays into the database. It uses "insert, or update if it's already there", so it's safe to run every time.

5. Rule 1 · No double booking (the most important thing you explain)

When do two bookings clash?

Two time windows overlap when **each one starts before the other ends**:

```
# a and b overlap  <=>  a.start < b.end  and  b.start < a.end
```

Example:

BOOKINGS	OVERLAP?	WHY
14:00–16:00 and 15:00–17:00	**yes**	14 < 17 and 15 < 16
14:00–16:00 and 16:00–18:00	**no**	the second starts exactly when the first ends

Back-to-back bookings are fine, because a booking holds the bay up to but *not including* its end.

Defence 1: check and save in one locked step (`insert_booking`, lines 488–509)

```
with transaction():                                  # BEGIN IMMEDIATE ... COMMIT
    clash = q_one(CLASH_SQL, bay_id, end_ms, start_ms)
    if clash:
        raise overlap_error(bay_id, clash)           # -> 409 Conflict, nothing saved
    INSERT INTO bookings ...
```

Why "locked" matters. Imagine two people press Hold for A-14 at 14:00 at the very same instant. Without a lock, this could happen:

1. Request 1 checks: "is A-14 free?" Yes.
2. Request 2 checks: "is A-14 free?" Yes.
3. Request 1 saves.
4. Request 2 saves. **Two bookings for one bay.**

This is called a **race condition**. We stop it two ways:

- **A Python lock** (`LOCK = threading.RLock()`, line 209). Only one thread touches the database at a time.
- **`BEGIN IMMEDIATE`** (line 242). It takes SQLite's write lock at the *start* of the transaction, so nothing else can write between our check and our save.

So request 2's check runs *after* request 1 has saved, sees the clash and gets **409 Conflict**.

A transaction means "everything inside happens completely, or not at all". If anything fails, it's rolled back (`ROLLBACK`, line 246).

Defence 2: a trigger inside the database (lines 169–189)

```
CREATE TRIGGER bookings_no_overlap_insert
BEFORE INSERT ON bookings WHEN NEW.status != 'cancelled'
BEGIN
  SELECT RAISE(ABORT, 'overlap') WHERE EXISTS (
    SELECT 1 FROM bookings b
    WHERE b.bay_id = NEW.bay_id AND b.status != 'cancelled'
      AND b.start_ms < NEW.end_ms AND NEW.start_ms < b.end_ms
  );
END;
```

A **trigger** is a rule the database runs by itself before every insert (and a second one before every update). Even if some future code forgot to check, or someone edited the database by hand, **the database itself refuses** an overlapping booking.

If the trigger fires, the server turns it into the same friendly 409 (lines 506–508).

Why two defences? Defence in depth. The code check gives a clear error message. The trigger is the safety net that can't be bypassed.

What the person sees

The server replies with status **409** and a message like *"A-14 is already booked 14:00–16:00. Pick another window."*, plus the clashing times. The site shows it in the booking panel. Nothing is saved and nothing is charged.

6. Rule 2 · Status from time

A bay's status is **never stored**. It's worked out every time, from the clock and that bay's bookings (`slot_state`, lines 347–363):

```
if current:   status = "mine" if current["mine"] else "ending" if current["end"] - now <=
ENDING_SOON else "taken"
elif nxt and nxt["start"] - now <= HOLD_AHEAD:   status = "mine" if nxt["mine"] else "held"
else:         status = "free"
```

STATUS	RULE
free	No booking now, and nobody due within 30 minutes.
taken (Booked)	A booking is running now.
ending (Ending soon)	The running booking ends within **20 minutes** (`ENDING_SOON`).
held (Reserved)	The next booking starts within **30 minutes** (`HOLD_AHEAD`).
mine (Yours)	The current or next booking was made from this browser.

Why compute it, not store it? A stored status goes stale the moment the clock moves, and you'd need a job to keep updating 96 bays. Computing it means it can never be wrong. The frontend uses **the same function**, so the map and the API always agree.

The 15-minute grace window. It's shown to the driver as "held until arrival + 15 minutes". Be honest: in the prototype it's a rule we display and promise. Automatically freeing a no-show needs check-in at the gate, which is on the roadmap.

7. Rule 3 · Live without refresh (Server-Sent Events)

How it works (lines 637–660 and 999–1021)

1. **The browser connects.** It opens `GET /api/lots/kiet-main/stream`, and the server **doesn't close the reply**. It sends the header `Content-Type: text/event-stream` and keeps the connection open.
2. **It's registered.** The server gives that page its own **inbox** (a `queue.Queue`) and adds it to `LISTENERS`, along with which lot it's watching and whose device it is.
3. **It's greeted.** The server sends `retry: 2000` (reconnect after 2 s if dropped) and a `hello` message.
4. **When something happens** (a booking, cancel, arrival or departure), `publish()` puts a `change` message into **every** inbox for that lot. Each page's thread wakes up and writes it out:

   ```
   event: change
   data: {"booking": {"bayId": "A-07", "start": …, "end": …, "mine": false, …}, "event": {…}}
   ```

5. **If nothing happens for 15 seconds,** it sends a `ping`, so the page knows the connection is still alive. The page reconnects if it hears nothing for 40 s.
6. **When the page closes,** writing fails (`OSError`), and the inbox is removed.

A detail judges like. Each message is **personalised**. Your own booking arrives with `mine: true`, your full plate and your code. Everyone else gets your plate masked and no code.

Why SSE, and not polling or WebSockets?

OPTION	HOW IT WORKS	WHY NOT
Polling	The page asks "anything new?" every few seconds.	Wasteful, and always a few seconds late.
WebSockets	A two-way connection.	Needs a separate protocol and more code. We only need server → browser.
SSE	One-way, server → browser, over normal HTTP.	**Built into every browser, reconnects by itself, and works with Python's standard library.**

Many people at once

- `ThreadingHTTPServer` gives **each request its own thread**, so one page's open stream doesn't block another person's booking.
- `daemon_threads = True` means those threads stop when the server stops.

8. Rule 4 · Private by default

No accounts (`identify`, lines 949–957)

1. On a browser's first visit, the server makes a random ID with `secrets.token_urlsafe(24)`: 32 random characters.
2. It sends that ID back as a cookie, `ps_device`. The cookie settings:

SETTING	MEANING
`HttpOnly`	JavaScript on the page can't read it.
`SameSite=Lax`	Other websites can't use it.
`Max-Age` = 1 year	It lasts a year.

3. The server **stores only** `sha256(device_id)`, called the **owner**. A hash is a one-way fingerprint. If the database ever leaked, nobody could turn the hash back into the cookie and pretend to be you.
4. Every request: hash the cookie, compare it with each booking's `owner`. If they match, that booking is **yours** (in any tab, after a refresh).

Showing data (`booking_out`, lines 616–624)

- **Your bookings:** full plate, your gate code, when you made it.
- **Everyone else's:** plate **masked** (`UP14 AB 4821` → `UP14 •• 4821`, `mask_plate`, line 366) and **no code**. So nobody can copy your code to the gate.

Booking codes (line 496)

- **Format:** `PS-` + 5 characters from `ABCDEFGHJKLMNPQRSTUVWXYZ23456789`. There's no `0/O` or `1/I`, so it's easy to read out loud.
- **How many:** 32 × 32 × 32 × 32 × 32 = **33,554,432** possible codes.
- **Randomness:** made with `secrets`, which is cryptographically secure, unlike `random`.
- **Uniqueness:** the column is `UNIQUE`. In the very rare case of a repeat, it tries again (up to 6 times).
- **"PS"** is left over from the project's first name, **ParkSeat**. The same goes for `parkseat.db`, the `ps_device` cookie and the `PARKSEAT_*` settings. We renamed the product to Arrivo on the site but kept the internal names, so existing bookings still work.

9. Making a booking: every check (`create_booking`, lines 543–572)

When the browser sends `POST /api/bookings` with `{ "bayId": "A-14", "start": …, "end": …, "plate": "UP14 AB 1234" }`:

CHECK	IF IT FAILS
The body is JSON with those fields	400 "Send bayId, start, end and plate as JSON."
The bay exists	404 "There is no bay called …"
Start and end are whole numbers	400 "Pick when you arrive and when you leave."
End is after start	400 "A booking has to end after it starts."
Length is **15 minutes to 12 hours**	400 "Bookings run from 15 minutes to 12 hours."
Start isn't in the past (2 minutes of slack)	400 "That start time has already passed."
Start is at most **2 days ahead**	400 "You can book up to two days ahead."
Plate: 6–14 letters, digits or spaces (cleaned and upper-cased first)	400 "Add your vehicle number, e.g. UP14 AB 1234."
Request body under 10 KB	413 "That request is too large."
No clash (locked check + trigger)	**409** "A-14 is already booked 14:00–16:00. Pick another window."

If everything passes:

1. Save the booking with a new code and the owner hash.
2. Record a `book` event for the feed.
3. `publish()` it to every open page.
4. Reply **201 Created** with the booking.

Never trust the browser. The site already checks most of these, but anyone can send requests without our site (with tools like `curl` or Postman). So the server checks everything again.

Cancelling (`cancel_booking`, lines 575–592)

- Only the device that made it can cancel. Anyone else gets **403**.
- **Before it starts:** status becomes `cancelled`.
- **While parked:** status becomes `released`, and the end moves to *now*, freeing the bay.
- This works on the server, but **there's no cancel button in the site yet**.

HTTP status codes we use

CODE	MEANING	OUR EXAMPLES
200	OK	snapshot, bay list
201	Created	a new booking
204	No content	browser pre-checks (CORS `OPTIONS`)
400	Bad request	bad time, bad plate, bad JSON
403	Forbidden	cancelling someone else's booking
404	Not found	no such bay, lot or URL
409	**Conflict**	**already booked**, or the booking already ended
413	Too large	body over 10 KB
500	Server error	a bug; the message is friendly and the details are printed for us

10. The background: clock, regular cars and demo cars

The clock, every 5 seconds (`clock_tick`, lines 670–687)

- Finds bookings that **started** since the last tick, records an `arrive` event and publishes it.
- Finds bookings that **ended**, records a `leave` event and publishes it.
- When the date changes, it adds that day's **regular cars** (below).
- Keeps only the latest **5,000** feed events.

The regular cars (`seed_plan`, lines 417–435)

So the lot looks like a real teaching day, the server adds **background traffic** once per date.

- **How busy each hour is:** a busyness table, `DEMAND`. It's about 40% overnight (hostel and staff cars) and 86–90% from 9:00 to 15:00.
- **Same date, same cars:** a **seeded random generator** (mulberry32, started from a hash of the date and the bay). That's why each date looks the same every time.
- **Made-up drivers:** plates like `UP14 KD 4821` from common local prefixes (UP14, UP16, DL3C, HR26…).

Demo cars (`demo_step`, lines 694–738)

Every **15–30 seconds**, but **only while someone has the map open**, one demo move happens:

- a car **arrives**
- a driver **reserves** for later
- a car **leaves early**

It leans towards how full the regular cars usually make the lot at that hour. Demo cars **never touch a real person's booking**.

Be honest about this in the demo. The cars moving around are background and demo traffic so the lot looks alive. **Bookings made on the site are real** (`source = 'api'`) and saved.

11. The web server (section 9)

Every URL (`ROUTES`, lines 882–893)

METHOD + URL	WHAT IT RETURNS
`GET /api/health`	`{"ok": true, ...}`: the deck's slide 10 checks this to see if the server is up.
`GET /api/lots`	The list of lots (just KIET for now) with bay counts.
`GET /api/lots/kiet-main/bays`	Every bay and its status right now, plus counts.
`GET /api/lots/kiet-main/snapshot?from=&to=`	**What the map loads:** every booking in the window (today by default) and the latest 30 feed events.
`GET /api/lots/kiet-main/stream`	**The live stream** (SSE).
`GET /api/bays/A-14/bookings`	One bay's status and bookings.
`POST /api/bookings`	**Make a booking.**
`DELETE /api/bookings/{id}`	Cancel or end your booking.
`GET /api/demo`, `POST /api/demo`	See or switch demo cars on or off.
anything else (`GET /`, `/assets/…`)	**The website** from `frontend/dist`.

How a request is handled (`handle_request`, lines 919–945)

1. Read the URL and query.
2. **`identify()`:** read or create the device cookie, and hash it.
3. **`allow_cors()`:** let `localhost` pages, plus any site listed in `CORS_ORIGIN`, call the API with cookies.
4. If the path doesn't start with `/api`, **serve the website**.
5. Otherwise, find the matching route and run it.
   - An `ApiError` becomes a JSON error with its status code.
   - Any other crash becomes a 500 with a friendly message, and the details are printed to the console for us.

Serving the website (`serve_site`, lines 1023–1035)

- **Files:** it sends files from `frontend/dist`. Any page that isn't a file gets `index.html`.

- **Security:** it checks the resolved path is still inside `dist`, so a URL like `/../../secret.txt` can't read other files (that attack is called *path traversal*).
- **Caching:** files in `/assets/` have a fingerprint in their name, so the browser may cache them for a year. `index.html` is never cached, so a new build shows up immediately.

Starting up (`main`, lines 1086–1117)

1. Open the database.
2. Sync the lot.
3. Run the clock once.
4. Start the server on port **8787**. It listens on IPv6 and IPv4. If the port is busy, it prints a clear message.
5. Start the two background loops: the clock every 5 s, and demo moves every 15–30 s.
6. Print the addresses.

12. Run it yourself

```
cd backend
python server.py
```

You'll see something like:

```
ParkSeat API on http://localhost:8787  |  96 bays  |  demo cars on
Database: ...\backend\data\parkseat.db
Site:     http://localhost:8787  (the built frontend)
```

COMMAND	WHAT IT DOES
`python server.py`	Start normally.
`python server.py --no-demo`	Start without demo cars.
`python server.py --port 8788`	Use another port.
`python server.py --reset-demo`	Remove today's demo cars (real bookings stay), then stop. Start it again after.
`python server.py --reset`	**Delete the whole database**, then stop. The next start is fresh. Careful: real bookings go too.
`python server.py --help`	List every option.

Try the API yourself. While it's running, open these in a browser:

- `http://localhost:8787/api/health`
- `http://localhost:8787/api/lots/kiet-main/bays`

- `http://localhost:8787/api/lots/kiet-main/stream`: watch the messages arrive. Pings come every 15 s, and changes come when cars move.

Every booking or cancellation is printed in the console, e.g. `14:02:31 POST /api/bookings -> 201`. During the demo, keep the console visible and point at it when the double booking gets **409**.

13. Honest limits

- **No real payment yet.** UPI is next.
- **No accounts.** A cleared cookie means the site no longer knows which bookings are yours.
- **No check-in at the gate yet.** So "reserved, waiting 15 minutes" is a promise shown to the driver, not yet enforced by scanning. QR check-in is next.
- **One process, one SQLite file.** That's perfect for one campus. For many cities you'd move to a database server (like PostgreSQL) and run several servers. The booking rules stay the same.
- **One lock for the whole database.** That's simple and safe at our size. At very large scale you'd lock per bay or let the database handle it.
- **The demo switch (`POST /api/demo`) has no password.** That's fine for a prototype on our own laptop; it would need an admin login before going public.
- **No per-person limit** on how many bays one device can book. It's an easy rule to add.
- **No automated tests yet.** We tested by hand, including two windows booking the same bay.

14. Questions to prepare

Basics

Q1. What's your backend built with? Python, using only its standard library: `http.server` for the server, `sqlite3` for the database, `threading` for many users at once, and Server-Sent Events for live updates. It's one file, `server.py`, with no `pip install`.

Q2. Why Python and not Node.js, Django or Flask? Python is easy to read and learn, and its standard library already has a web server and a database. For a prototype, one readable file with nothing to install beats a framework we'd have to learn and configure. Flask or Django would be the step up for a bigger product.

Q3. Why SQLite and not MySQL or MongoDB?

- It's one file with no separate database server, and it's built into Python.
- It supports transactions and triggers, which is exactly what we need to make double booking impossible.
- Our data is naturally tables (lots, bays, bookings), so a relational database fits.
- For one campus it's fast enough. To grow, we'd move to PostgreSQL.

Q4. What is an API? The set of URLs our server answers. The website calls them to load the lot, make a booking, and listen for changes. They talk in JSON.

Q5. What is JSON? A text format for data: `{"bayId": "A-14", "start": 1792573200000}`. Both JavaScript and Python read and write it easily.

Double booking

Q6. How do you stop two people booking the same bay? Twice. First, the server checks for a clash and saves the booking in **one locked transaction**, so two requests can't both pass the check. Second, a **database trigger** refuses any two overlapping bookings on one bay, whoever writes them. The second person gets a 409 Conflict.

Q7. What's a race condition? When two things happen at the same time and the result depends on which goes first. Here: two requests both check "is it free?" before either saves, so both save. The lock and `BEGIN IMMEDIATE` force them to go one after the other.

Q8. What's a transaction? A group of database steps that happen completely or not at all. If anything fails in the middle, everything is undone (rolled back).

Q9. What's a trigger? A rule stored inside the database that runs automatically before (or after) an insert or update. Ours checks for overlaps and aborts the insert if there is one.

Q10. How do you decide two bookings overlap? Each starts before the other ends: `a.start < b.end and b.start < a.end`. Back-to-back is allowed: one ending at 16:00 and the next starting at 16:00 don't clash.

Q11. Why 409? It's the standard HTTP code for "conflict with the current state", which is exactly what an already-booked bay is.

Live updates

Q12. How do other screens update without refreshing? Each open page keeps one connection to the server, using Server-Sent Events. When a booking is saved, the server writes a message into every open page's inbox, and each page updates that one bay.

Q13. Why not WebSockets? We only need the server to talk to the browser, not back. SSE is simpler, built into browsers, reconnects by itself, and works with Python's standard library. Bookings themselves go through a normal POST.

Q14. What happens if the connection drops? The page reconnects automatically and reloads the day, so nothing is missed. The server pings every 15 seconds; if the page hears nothing for 40 seconds, it reconnects.

Q15. How many users can it handle? Each page uses one thread and one connection. That's fine for a campus: dozens to a few hundred at once. For thousands, we'd move to an async server and a database server. The rules stay the same.

Privacy and security

Q16. How do you know which booking is mine without a login? Your browser gets a random ID in a cookie the first time. We store only its SHA-256 hash with your bookings. When you come back, we hash your cookie again and compare.

Q17. Why store a hash and not the ID? A hash is one-way. If the database leaked, nobody could get your cookie back from it and pretend to be you.

Q18. Can someone see my number plate? Only you see it in full. Everyone else sees it masked, like `UP14 •• 4821`, and nobody else ever receives your gate code.

Q19. Can someone guess a booking code? There are about 33.5 million possible codes, made with Python's `secrets` module (secure randomness), and other people's codes are never sent out. A real gate would also check the plate.

Q20. What if someone sends fake requests without your site? The server checks everything again: bay, times, length, plate and clash. It doesn't trust the browser. It also caps request size at 10 KB.

Q21. Is the server secure enough to go live? For a prototype on our laptop, yes. Before going public we'd add HTTPS, an admin login for the demo switch, rate limits, and per-person booking limits.

Data and demo

Q22. Are those real cars? The bays are real. The moving cars are background and demo traffic the server makes up so the lot looks alive. The demo runs only while someone has the map open, and it never touches a real booking. Any booking made on the site is real and saved.

Q23. Why store times as milliseconds? Because JavaScript's `Date.now()` uses milliseconds since 1970. The browser and server use the same numbers, with no date-format or timezone confusion between them.

Q24. Why is a bay's status not stored in the database? It depends on the time, so a stored value would go stale every minute. We compute it from the bookings and the clock, using the same function as the frontend, so it's always right and both sides agree.

Q25. Why does the code say ParkSeat? That was the project's first name. We renamed the product to Arrivo but kept internal names (the database file, cookie, `PS-` codes) so existing bookings keep working.

Q26. What was hardest? Making double booking truly impossible even at the same instant, and keeping every open screen in sync, including after a dropped connection.

Q27. Did you use AI? Answer truthfully, the same way as the rest of the team. If you did, say so plainly and show you understand the result. For example: "Yes, we used AI tools while building. We tested the rules ourselves, including two windows booking the same bay, and I can walk you through every section of `server.py`."

15. Words you should know

WORD	MEANING
Server	A program that waits for requests and answers them.
Port	A numbered "door" on a computer. Ours is 8787.
Request / response	What the browser asks / what the server answers.
HTTP method	The kind of request: GET (read), POST (create), DELETE (remove).
Endpoint / route	One URL the API answers, e.g. `POST /api/bookings`.
Status code	The number in a reply: 200 OK, 201 created, 400 bad, 404 not found, 409 conflict.
JSON	The text format data travels in.
Database / table / row / column	Where data lives / a grid of one kind of thing / one item / one field.
Primary key	The column that uniquely identifies a row (`id`).
Foreign key	A column that must point at a row in another table (a booking's `bay_id`).
Index	A lookup structure that makes searches fast.
Transaction	Steps that happen completely or not at all.
Trigger	A rule the database runs by itself before or after a change.
Race condition	A bug when two things happen at once and the order matters.
Lock	Makes others wait until you're done.

WORD	MEANING
Thread	One line of work inside a program; many can run at once.
SSE	Server-Sent Events: a long-lived connection where the server pushes messages.
Cookie	A small value the browser stores and sends with each request.
Hash (SHA-256)	A one-way fingerprint of some text; it can't be reversed.
CORS	Browser rules about which websites may call your API.
Seeded random	Random numbers you can repeat exactly by starting from the same seed.
Standard library	The modules that come with Python; no install needed.

16. Numbers to remember

File	`backend/server.py` · **1,121 lines** · 9 sections · Python 3.8+ · **0** pip installs
Address	`http://localhost:8787` (site + API)
Database	SQLite · `backend/data/parkseat.db` · 6 tables · WAL mode
Bays	**96** in **5 rows** (A 33 · B 10 · C 24 · D 6 · E 23)
Rates	₹15 compact · ₹20 standard / step-free · ₹35 EV (per hour)
Booking length	**15 min to 12 h** · up to **2 days ahead** · 2 min slack for "now"
Status rules	ending ≤ **20 min** · reserved = due within **30 min** · grace **15 min**
Codes	`PS-` + 5 chars from 32 → **33,554,432** possibilities
Device ID	24 random bytes (32 characters) · stored as **SHA-256** · cookie lasts **1 year**
Live	ping every **15 s** · browser retry **2 s** · page watchdog **40 s**
Background	clock every **5 s** · demo move every **15–30 s** (only when watched) · feed keeps **5,000** events
Limits	request body **10 KB** · plate **6–14** chars